

AI WEBSITE BUILDER

Complete Specification Document

Build a Free, Scalable AI-Powered Website Generator

100% Free to Launch
Scalable to 100+ Concurrent Users
Multi-API Architecture (Z.AI + OpenRouter + Groq)
Session-Based API Locking System
Figma Design Import Support

1. Executive Summary

This document provides a complete specification for building an AI-powered website builder platform that can be launched at zero cost and scaled to handle 100+ concurrent users. The platform will compete with services like Lovable.dev, Bolt.new, Cursor, and Replit, but with a unique value proposition: completely free to start with a sustainable scaling model.

1.1 Core Value Proposition

The platform enables users to generate complete websites from text prompts or Figma designs using free AI APIs. Users can preview their sites in real-time and deploy them to their own Vercel accounts. The business model is based on the 'earn first, pay later' principle - the platform uses free API tiers initially and scales to paid APIs only when revenue is generated.

1.2 Key Differentiators

- 100% free to launch - no upfront costs for API access
- Multi-API architecture ensures redundancy and scalability
- Session-based API locking preserves context and saves tokens
- Figma design import with 99.99% design fidelity
- User deploys to their own Vercel account (no hosting costs)
- Template marketplace for community-driven revenue

2. Account Creation Guide

To achieve 10+ concurrent API requests, you will need to create accounts across multiple AI API providers. Each account provides access to free tier APIs that can be pooled together for concurrent execution.

2.1 Z.AI Account Setup

Z.AI provides free access to GLM-4.7-Flash and GLM-4.5-Flash models with no credit card required. Each account can handle 1 concurrent request.

Step	Action	URL
1	Visit Z.AI main website	https://z.ai
2	Click 'Sign Up' or 'Register'	https://z.ai/register
3	Create account with email or social login	-
4	Navigate to API Keys management	https://z.ai/manage-apikey/apikey-list
5	Click 'Create API Key'	-
6	Copy and save the API key securely	-

2.2 OpenRouter Account Setup

OpenRouter provides unified access to 50+ free AI models from various providers including Z.AI, DeepSeek, Meta, Google, and more. One account gives access to multiple free models.

Step	Action	URL
1	Visit OpenRouter	https://openrouter.ai
2	Click 'Sign In' or 'Sign Up'	https://openrouter.ai/sign-in
3	Sign up with GitHub, Google, or email	-
4	Go to Keys section in dashboard	https://openrouter.ai/keys
5	Create a new API key	-
6	Copy and save the API key	-

2.3 Groq Account Setup

Groq provides ultra-fast AI inference with free tier access. Rate limits: 30 RPM, 14,400 requests/day, 6K TPM. Best for fast fallback responses.

Step	Action	URL
1	Visit Groq Console	https://console.groq.com
2	Sign up with Google, GitHub, or email	https://console.groq.com/login
3	Navigate to API Keys section	https://console.groq.com/keys
4	Create new API key	-
5	Copy and save the API key	-

2.4 Recommended Account Strategy

Provider	Accounts Needed	Concurrent Requests	Free Models
Z.AI	5-10	5-10	GLM-4.7-Flash, GLM-4.5-Flash
OpenRouter	3-5	3-5	50+ free models
Groq	2-3	2-3	Llama, Mixtral
Total	10-18	10-18 concurrent	-

3. API Reference

3.1 Z.AI REST API

Base URL: <https://api.z.ai/api/paas/v4>

Chat Completion Endpoint:

POST <https://api.z.ai/api/paas/v4/chat/completions>

Headers:

Content-Type: application/json

Authorization: Bearer YOUR_API_KEY

Body:

```
{
  "model": "GLM-4.7-Flash",
  "messages": [
    {"role": "system", "content": "You are an expert web developer."},
    {"role": "user", "content": "Create a landing page..."}
  ],
  "temperature": 0.7,
  "max_tokens": 8192
}
```

3.2 OpenRouter API

Base URL: <https://openrouter.ai/api/v1>

Free Models Available:

Model ID	Provider	Best For
z-ai/glm-4.5-air:free	Z.AI	General coding
deepseek/deepseek-r1:free	DeepSeek	Reasoning
google/gemini-2.0-flash-exp:free	Google	Fast responses
meta-llama/llama-4-scout:free	Meta	General tasks
qwen/qwen-2.5-72b-instruct:free	Qwen	Long context
openrouter/free	OpenRouter	Random free model

3.3 Groq API

Base URL: <https://api.groq.com/openai/v1>

Groq uses OpenAI-compatible endpoints. Free tier includes access to Llama-4, Mixtral, and other models with 30 RPM rate limit.

4. Session-Based API Locking System

4.1 Core Principle

The session-based API locking system ensures that once a user starts a conversation with an AI model, the same API key remains locked to that user for 22 minutes of inactivity. This preserves conversation context and saves tokens by avoiding re-sending the entire conversation history.

4.2 Architecture Overview

User Request Flow:

1. User sends prompt -> Check for existing session
2. If no session -> Acquire available API key from pool
3. Lock API key to user session
4. Maintain full conversation context
5. Release API key after 22 minutes of inactivity
6. Archive session data for potential resume

4.3 API Pool Manager Code

```
// lib/api-pool.ts
interface APIKey {
  key: string;
  inUse: boolean;
  lastUsed: number;
  requestCount: number;
  owner: string;
}

class ZAIAPIPool {
  private apiKeys: APIKey[] = [];
  private queue: Array<{
    prompt: string;
    resolve: Function;
    reject: Function;
  }> = [];

  constructor(keys: string[]) {
    this.apiKeys = keys.map((key, index) => ({
      key,
      inUse: false,
      lastUsed: 0,
      requestCount: 0,
      owner: `account_${index + 1}`
    }));
  }

  async generateCode(prompt: string): Promise<string> {
    const availableKey = this.apiKeys.find(k => !k.inUse);

    if (!availableKey) {
      return new Promise((resolve, reject) => {
        this.queue.push({ prompt, resolve, reject });
      });
    }

    availableKey.inUse = true;
    availableKey.lastUsed = Date.now();
```

```

try {
  const result = await this.callAPI(availableKey.key, prompt);
  availableKey.requestCount++;
  return result;
} finally {
  availableKey.inUse = false;
  this.processQueue();
}
}

private async callAPI(apiKey: string, prompt: string): Promise<string> {
  const response = await fetch(
    'https://api.z.ai/api/paas/v4/chat/completions',
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${apiKey}`
      },
      body: JSON.stringify({
        model: 'GLM-4.7-Flash',
        messages: [
          { role: 'system', content: 'You are an expert web developer.' },
          { role: 'user', content: prompt }
        ],
        temperature: 0.7,
        max_tokens: 8192
      })
    }
  );

  const data = await response.json();
  return data.choices[0].message.content;
}
}

```

4.4 Session Manager Code

```

// lib/session-manager.ts
interface Session {
  id: string;
  userId: string;
  apiKey: string;
  apiProvider: 'openrouter' | 'zai' | 'groq';
  messages: Message[];
  codeState: string;
  lastActivity: number;
  locked: boolean;
  createdAt: number;
}

class SessionManager {
  private sessions: Map<string, Session> = new Map();
  private readonly INACTIVITY_TIMEOUT = 22 * 60 * 1000; // 22 minutes

  constructor() {
    setInterval(() => this.checkExpiredSessions(), 60000);
  }

  async createSession(userId: string): Promise<Session> {
    const apiKey = await this.apiPool.acquire();

    const session: Session = {

```

```

        id: generateUUID(),
        userId,
        apiKey: apiKey.key,
        apiProvider: apiKey.provider,
        messages: [],
        codeState: "",
        lastActivity: Date.now(),
        locked: true,
        createdAt: Date.now()
    };

    this.sessions.set(session.id, session);
    return session;
}

updateActivity(sessionId: string): void {
    const session = this.sessions.get(sessionId);
    if (session) {
        session.lastActivity = Date.now();
    }
}

private checkExpiredSessions(): void {
    const now = Date.now();

    for (const [sessionId, session] of this.sessions) {
        const inactiveTime = now - session.lastActivity;

        if (inactiveTime >= this.INACTIVITY_TIMEOUT) {
            this.apiPool.release(session.apiKey);
            this.archiveSession(session);
            this.sessions.delete(sessionId);
        }
    }
}

```

5. Figma Design Import System

5.1 Overview

The Figma design import system enables users to convert their Figma designs into production-ready websites with 99.99% design fidelity. The system uses a custom Figma plugin that exports all design elements including images, assets, colors, fonts, and layout information in a structured JSON format.

5.2 Key Requirement: No AI-Generated Images

IMPORTANT: The AI should NEVER generate images for Figma-imported designs. All images, icons, and assets must come directly from the Figma design export. This ensures 99.99% design fidelity and respects the designer's original work.

5.3 Export Capabilities

Element Type	Export Format	Notes
Images	Base64 / PNG / SVG	Full resolution preserved
Colors	Hex codes	Primary, secondary, accent colors
Fonts	Font family, size, weight	Google Fonts mapping
Layout	CSS Flexbox/Grid	Responsive breakpoints
Spacing	Pixels / Rem	Margin, padding, gap values
Assets	Binary / Base64	Icons, logos, illustrations
Effects	CSS properties	Shadows, borders, opacity

5.4 Recommended Figma Plugins

Plugin	Purpose	URL
Export Kit	Design to JSON export	figma.com/community/plugin/1532383276961566436
Design-to-JSON	Full design export	figma.com/community/plugin/1514601930647701205
Base64	Image to Base64	figma.com/community/plugin/1267577596388638989
Bulk Image Exporter	Asset extraction	figma.com/community/plugin/1546130574483909562
Builder.io	Design to code	figma.com/community/plugin/747985167520967365

5.5 JSON Export Structure

```
{
  "design": {
```



```
"name": "My Website Design",
"width": 1440,
"height": 900,
"frames": [
  {
    "id": "frame-1",
    "name": "Hero Section",
    "type": "FRAME",
    "x": 0, "y": 0,
    "width": 1440, "height": 600,
    "styles": {
      "backgroundColor": "#FFFFFF",
      "padding": { "top": 80, "bottom": 80 }
    },
    "children": [
      {
        "type": "TEXT",
        "content": "Welcome to My Site",
        "styles": {
          "fontFamily": "Inter",
          "fontSize": 48,
          "fontWeight": 700,
          "color": "#1A1A1A"
        }
      },
      {
        "type": "IMAGE",
        "src": "data:image/png;base64,...",
        "width": 200,
        "height": 200,
        "alt": "Hero Image"
      }
    ]
  }
],
"tokens": {
  "colors": {
    "primary": "#2563EB",
    "secondary": "#64748B",
    "background": "#FFFFFF"
  },
  "fonts": {
    "heading": "Inter",
    "body": "Inter"
  },
  "spacing": {
    "unit": 8
  }
},
"assets": [
  {
    "name": "logo.png",
    "type": "image/png",
    "data": "base64_encoded_data"
  }
]
}
```

6. Environment Configuration

6.1 Required Environment Variables

```
# .env.local

# === Z.AI API Keys Pool (5-10 keys recommended) ===
ZAI_KEY_1=your_zai_api_key_1
ZAI_KEY_2=your_zai_api_key_2
ZAI_KEY_3=your_zai_api_key_3
ZAI_KEY_4=your_zai_api_key_4
ZAI_KEY_5=your_zai_api_key_5
# Add more keys as needed...

# === OpenRouter API Keys (3-5 recommended) ===
OPENROUTER_KEY_1=your_openrouter_key_1
OPENROUTER_KEY_2=your_openrouter_key_2
OPENROUTER_KEY_3=your_openrouter_key_3

# === Groq API Keys (2-3 recommended) ===
GROQ_KEY_1=your_groq_key_1
GROQ_KEY_2=your_groq_key_2

# === Database (Supabase Free Tier) ===
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key

# === Vercel Deployment ===
VERCEL_TOKEN=your_vercel_token
VERCEL_ORG_ID=your_org_id
VERCEL_PROJECT_ID=your_project_id

# === Application Settings ===
SESSION_TIMEOUT_MINUTES=22
MAX_QUEUE_WAIT_SECONDS=30
CIRCUIT_BREAKER_THRESHOLD=5
```

6.2 Supabase Setup

Supabase provides free PostgreSQL database with authentication. Create account at: <https://supabase.com>

Database Schema:

```
-- Users table (managed by Supabase Auth)
-- Projects table
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  name VARCHAR(255),
  description TEXT,
  code TEXT,
  figma_json JSONB,
  vercel_deployment_id VARCHAR(255),
  vercel_url VARCHAR(500),
  status VARCHAR(50) DEFAULT 'draft',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Sessions table
```

```
CREATE TABLE sessions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES auth.users(id),  
  project_id UUID REFERENCES projects(id),  
  api_key_id VARCHAR(50),  
  messages JSONB DEFAULT '[]',  
  last_activity TIMESTAMP DEFAULT NOW(),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- API Keys pool (admin only)

```
CREATE TABLE api_keys (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  provider VARCHAR(50),  
  key_ref VARCHAR(100),  
  daily_limit INTEGER,  
  daily_used INTEGER DEFAULT 0,  
  is_active BOOLEAN DEFAULT true  
);
```

7. Deployment Strategy

7.1 Vercel Free Tier Limits

Resource	Free Tier Limit	Notes
Bandwidth	100 GB/month	Sufficient for ~50K users
Serverless Invocations	1,000,000/month	~100 concurrent users
Build Minutes	Unlimited	45 min max per build
Deployments	3,000/day	More than enough
Serverless Functions	12 per deployment	Use API routes instead
Execution Timeout	10 seconds	Hobby plan limit

7.2 User Deployment Flow

Users deploy generated websites to their own Vercel accounts. This eliminates hosting costs for the platform and gives users full ownership of their projects.

User Deployment Process:

1. User creates a Vercel account (free) at vercel.com
2. User generates a Vercel API token
3. User connects the token to your platform
4. Your platform creates deployments on behalf of the user
5. User connects their custom domain (optional)
6. All hosting costs are borne by the user's Vercel account

Vercel API Integration:

POST <https://api.vercel.com/v13/deployments>

Headers:

Authorization: Bearer USER_VERCEL_TOKEN

Content-Type: application/json

Body:

```
{
  "name": "project-name",
  "files": [...],
  "projectSettings": {
    "framework": "nextjs"
  }
}
```

8. Development Roadmap

8.1 Phase 1: MVP (Week 1-2)

- Landing page with authentication (Supabase Auth)
- Basic AI code generation using Z.AI API
- Live preview sandbox (iframe-based)
- Vercel deployment integration
- Session management with 22-minute timeout
- API pool manager for 10 concurrent requests

8.2 Phase 2: Core Features (Week 3-4)

- Template gallery (10 starter templates)
- Multi-API fallback system (Z.AI + OpenRouter + Groq)
- User dashboard with project management
- Code editor with live preview
- Project saving and resumption

8.3 Phase 3: USP Features (Week 5-6)

- Figma plugin development
- Figma JSON import functionality
- Design token extraction
- Asset export and optimization
- 99.99% design fidelity guarantee

8.4 Phase 4: Scale (Month 2+)

- SEO optimization for SERP capture
- Template marketplace launch
- Revenue integration (Stripe)
- Premium features implementation
- Community features

9. Master Development Prompt

The following is the complete prompt to paste into an AI assistant to build this platform:

Build a complete AI-powered website builder platform with these specifications:

ARCHITECTURE:

- Next.js 16 with TypeScript and Tailwind CSS 4
- Supabase for authentication and database (free tier)
- Multiple free AI API providers: Z.AI, OpenRouter, Groq
- Deploy on Vercel free tier

CORE FEATURES:

1. User Authentication: Supabase Auth with email/social login
2. AI Code Generation: Multi-API pool with session locking
3. Live Preview: Sandpack or iframe sandbox for code preview
4. Vercel Deployment: User connects their Vercel account for deployment

SESSION-BASED API LOCKING:

- One API key = one active session
- 22-minute inactivity timeout before releasing API key
- Full conversation context preserved during session
- Queue system for overflow requests

API POOL MANAGER:

- Support 10+ concurrent API requests
- Round-robin key distribution
- Health check for each API key
- Circuit breaker for failed requests

FIGMA IMPORT:

- JSON-based design export
- Extract all images as base64
- Preserve colors, fonts, spacing
- AI converts JSON to production code
- NO AI-generated images - use only Figma assets

USER FLOW:

1. User writes prompt or uploads Figma JSON
2. AI generates website code
3. Live preview shows result
4. User iterates with chat
5. User deploys to their Vercel account

DESIGN YOUR OWN SITE:

- Use Figma to design your platform's UI
- Export via plugin as JSON
- Use the same import system for your own site
- Customize freely with your design

TECHNICAL STACK:

- Frontend: Next.js 16, React, TypeScript, Tailwind CSS
- Backend: Next.js API Routes (serverless functions)
- Database: Supabase PostgreSQL
- Auth: Supabase Auth
- AI: Z.AI, OpenRouter, Groq free tiers
- Deployment: Vercel

Create a production-ready, scalable platform that can handle 100+ concurrent users on free infrastructure.